Test Plan

MSDS 498 Capstone — NU High School Data Platform

Week 3 Deliverable — 2026-07-17

This plan covers unit, integration, system, and user acceptance testing for the NU high-school data platform (Sections 3-4 of the capstone research paper). It is grounded in the project's actual risk areas rather than a generic template: every scenario below traces to a specific, real risk already found in this pipeline, and several trace to actual bugs identified and fixed during development. A runnable test suite backing this plan lives in the project repository under tests/ (57 automated tests, all passing as of this writing — 55 unit/integration/CSV-system tests plus 2 that bring up the real Docker/Postgres pipeline end to end). A remaining coverage gap is stated plainly in Sections 1 and 4, rather than smoothed over, consistent with this project's standing practice of reporting uncomfortable findings rather than presenting only favorable ones.

1. Test Coverage

Four testing types, matched to where risk actually concentrates in this project:

Type	What it covers	Why it matters here
Unit	Individual functions in build_features.py, combine_schools.py, build_rigor_classification.py, build_modeling_dataset.py	This is where the real bugs have lived: sector misclassification, IB match-tier gating, LEAID derivation, the CEEB fan-out — all were logic errors inside single functions, catchable in isolation
Integration	How the CSV-driven pipeline stages compose (feature build -> cleaning freeze)	Catches column-contract breaks between stages that unit tests on either side, run alone, cannot see
System	The five-script CSV pipeline end to end, and the full Docker/Postgres pipeline against a freshly started database	Confirms the whole thing actually runs, not just that each piece is individually correct; this is also where a missing scikit-learn dependency was caught
User Acceptance (UAT)	Planned reviews of the data dictionaries, the modeling dataset, and the rigor/clustering/benchmarking outputs, by the project client and, per the assignment's guidance, a second student team	The platform's entire purpose is serving admissions officers standardized, auditable context — correctness by our own tests is necessary but not sufficient

Verified 2026-07-17: the Docker/Postgres pipeline works

docker compose up -d db, then docker compose run --rm etl, completed successfully in roughly 106 seconds against the raw source data. The resulting schools_org_all table's CEEB match count (16,508) matched the CSV-path result exactly, cross-validating both paths compute the same result from the same join logic. This is now covered by an automated test, tests/test_docker_pipeline.py, which skips (not fails) when Docker isn't reachable or the raw data isn't present — the same pattern the CSV-only system tests already used.

One gap remains — not undocumented anymore, but still not automatic

The reproducibility gap for a genuinely fresh clone is real, and running the pipeline once locally doesn't remove it. The raw source data this database-backed path needs — Sheng's combined schools export, the client's organizational export, and the raw CRDC/EDFacts assessment data — is gitignored, correctly: the directory totals roughly 2.6 GB (a single CRDC folder alone is 794 MB; two individual EDFacts files are 938 MB and 875 MB, each on its own over GitHub's 100 MB per-file limit), far past what is reasonable to version in git — checked directly against that limit before attempting anything, rather than found out via a failed push. As of 2026-07-17, this is a documented manual step, not an unwritten gap: the project README now links the team's Google Drive folder holding this data and says to download it before running the Docker path. A new team member or CI runner cloning this repository fresh still has to do that download

manually — the Docker test will correctly skip until they do — but the hand-off itself is written down. A small sanitized fixture remains a nice-to-have to eventually remove even that manual step in a CI context, not built yet.

combine_schools.py's SQL-backed join functions are now exercised end to end by the Docker system test and its row-count cross-check — a real improvement over having zero coverage — but this is system-level ("did the whole run succeed with the right row count"), not unit-level coverage of each function's specific join semantics. Dedicated unit tests for these functions, isolated from a live database, remain a gap.

Explicitly not covered by this plan (documented, not silently skipped):

- Load/performance testing — out of scope; this is a batch ETL pipeline re-run at most annually, not a service under concurrent load.
- Security testing — the platform has no external-facing API or auth surface yet; revisit if one is added.

2. Test Scenarios

Unit test scenarios (by module)

build_features.py

- Bucket-midpoint parsing across every observed format: exact ranges, open-ended high ("Over 90%", "greater than 20"), open-ended low ("10% or fewer"), bare numbers, unparseable strings, and null input.
- Winsorization clips extreme outliers at the 1st/99th percentile without crashing on all-null input.
- Sector classification: public HS, private HS via nu_type, private HS via a school-side pss_id record with no nu_type (the exact case a real bug missed), an org-only row with no school-side match, and mutual exclusivity of public/private flags.
- IB flag gating: review-tier match counts as a candidate, reject-tier does not (a real bug counted both); no-match rows are 0, not null.
- LEAID derivation: correctly takes the district ID from the 12-digit school ID, returns null when that ID is itself null (never falls back to a broken legacy column).

combine_schools.py

- Name normalization: standard abbreviation expansion, punctuation stripping, null-safe.
- CEEB tie resolution: best-tier match keeps the CEEB, the loser gets nulled and flagged; ties at equal tier prefer an exact name match; a frame with no duplicates is untouched; a unique CEEB is never modified.
- (SQL-backed join functions covered at the system level via the Docker test, not individually unit-tested — see Section 1's remaining gap.)

build_rigor_classification.py

- Z-scoring: mean-zero/std-one on normal input, all-NaN on zero-variance input (must not divide by zero), NaN passthrough.
- Weighted composite: matches a plain weighted average under full coverage; reallocates weight proportionally when a component is missing (not zero-imputed, not row-dropped); yields NaN (not 0) when zero components are available; a zero-weight component never influences the score even when present.
- Tier assignment: five roughly-equal quintile buckets, correct ordinal ordering, NaN scores stay untiered rather than defaulting to a tier.

build_modeling_dataset.py

- Minimum-size freeze: drops schools below the enrollment floor, keeps schools with unknown enrollment rather than treating null as "too small," boundary value is inclusive.
- Universe restriction: only public/private HS rows survive.
- Sentinel scrub: negative sentinel codes in rate/score-named columns become null; a legitimate zero is untouched; a non-suspect column name is never scrubbed.

Integration test scenarios

- A synthetic fixture (covering public HS, private-via-pss_id, private-via-nu_type, and org-only rows) survives the full feature-build-to-freeze chain with sector labels intact and no row collapsing into another.
- No duplicate school identities emerge from chaining these steps (a stand-in for the class of bug the real CEEB fan-out was).

System test scenarios (automated, verified)

- Each of the five CSV pipeline scripts runs to completion against the real exported data and produces output of the expected shape (row-count floor, required columns present, categorical outputs constrained to their valid value sets). Verified: all five pass as of this writing.
- The Docker/Postgres stack builds, runs the full ETL orchestration script successfully, and produces a schools_org_all row count matching the CSV-path result. Verified: both pass, completing in roughly 106 seconds. Only runs when Docker is reachable and the raw data is present locally — auto-skips otherwise, since neither holds on a fresh clone.

Remaining gap — not this pipeline's correctness, but who can run it

- The Docker/Postgres system test above is real and passing, but only on machines that already have the raw source data present locally (correctly gitignored, not a bug — see Section 4). A fresh clone, or a CI runner, still can't run it without a manual download step first — now documented in the project README via a linked team Drive folder, not an unwritten gap, but still a step a human has to do rather than something automated. A small sanitized fixture remains a nice-to-have to remove even that manual step in a CI context, not built yet.

UAT scenarios (planned — none of these have happened yet)

- The client can locate the source, vintage, and description for any variable they ask about in the data dictionaries — this is literally what was requested in an early project meeting.
- A reviewer unfamiliar with the pipeline can open the modeling dataset and correctly interpret key fields from the data dictionary alone, without reading the ETL code.
- A second student team, engaged per the assignment's UAT guidance, can independently reproduce one full pipeline run and get matching results, without live walkthrough support from our team.

3. Test Case Design

Full test cases live as executable code in the project's tests/ directory, not just described here, so they cannot drift out of sync with the actual pipeline. Representative examples, in steps / input / expected-result format:

ID	Test	Input	Steps	Expected Result
UT-01	Private-school sector classification	Fixture row: school-side pss_id record, no nu_type, no school_level	Run the feature-build function	is_private_hs = True, sector = "private" (regression test for a real bug where such rows were misclassified)
UT-02	IB reject-tier exclusion	Fixture row: IB match present, tier = "reject"	Run the feature-build function	ib_flag_candidate = 0 (regression test for a real bug that counted reject-tier matches as confirmed)
UT-03	Weighted composite with a missing component	3-component input, one component null for one row	Call the weighted-composite function	Score is computed from the remaining components with weight renormalized to sum to 1
UT-04	CEEB tie resolution keeps the best match	Two schools sharing one CEEB: one auto-accept exact-name match, one review-tier different school	Call the CEEB tie-resolution function	Auto-accept row keeps the CEEB; review-tier row's CEEB is nulled and flagged
IT-01	Full feature-to-freeze chain	4-row fixture (public HS, 2 private HS variants, 1 org-only row)	Run feature build, then universe restriction, then size freeze	Only public/private rows remain; each is public XOR private; the pss_id-only private school survived the chain
ST-01	Rigor classification system run (CSV	Real modeling dataset (34,392	Run the rigor-classification script	Exit code 0; output has a tier label

	path)	rows)	via subprocess	column; every non-null value is one of the five valid tier names
ST-02	Full Docker/Postgres pipeline	Raw source data present locally (~2.6 GB, gitignored — not present on a fresh clone)	Bring up Postgres via Docker Compose, wait healthy, run the ETL orchestration script	Exit code 0; "Pipeline completed successfully" in output. Auto-skips if Docker or the raw data isn't present, so it's automated where its precondition holds rather than manual everywhere. Verified passing 2026-07-17.
ST-03	Cross-path row-count validation	Same as ST-02	Query schools_org_all's row count from the live database after ST-02	Matches the CSV-path row count (53,966) — cross-validates both paths independently. Verified passing 2026-07-17.

Convention for adding new test cases: any newly found bug gets a regression test named for the bug it prevents, not just the function it touches — matching the pattern used throughout the existing suite.

4. Test Environment Setup

Two supported environments, matching how this pipeline is actually run day to day:

Environment	Description
CSV-only (no database)	What most contributors use day to day. Requires Python 3.11+ and the packages pinned in the ETL requirements file. Operates directly on the exported CSVs; this is what the CSV system tests run against.
Full Docker stack	docker-compose.yml brings up Postgres 16 and an ETL container that runs the full pipeline against it. Needed for the SQL-backed combination scripts and anyone re-deriving the exported CSVs from raw source data. Verified working end to end 2026-07-17 — see the callout below for the one real limitation that remains.

Verified working end-to-end 2026-07-17 — docker compose up -d db, then docker compose run --rm etl, completed successfully in roughly 106 seconds and produced a schools_org_all row count matching the CSV path exactly. Covered by an automated test, tests/test_docker_pipeline.py.

Resolved 2026-07-17 — the hand-off is documented, not just recommended. The raw-data directory this database-backed path needs — Sheng's combined schools export, the client's organizational export, and the raw CRDC/EDFacts assessment data — is gitignored, correctly: the directory totals roughly 2.6 GB (CRDC data alone is 794 MB; two individual EDFacts files are 938 MB and 875 MB, each over GitHub's 100 MB per-file limit on its own), far past what is reasonable to version in git — checked directly against that limit before attempting anything. Documented instead via a team Google Drive folder, linked from the project README's Quickstart section (request access from the team if it is not visible). A new contributor or CI runner starting from a fresh clone still has to download that data manually before this environment will do anything beyond auto-skip — that manual step is real and still required — but it is now a written-down three-minute setup step rather than an undocumented gap. A small sanitized fixture remains a nice-to-have to eventually remove even that manual step for a CI context, not built yet.

Test-specific dependencies are pinned in a dedicated requirements file (pytest), kept separate from the pipeline's runtime requirements file since pytest is a development/test-time dependency, not a production one.

A real environment gap this test plan already caught: the ETL requirements file was missing scikit-learn, which the clustering script requires — meaning the documented Docker environment could not actually run that script. Fixed as part of writing this plan — confirmed fixed by the successful Docker build above, not just asserted.

Configuration: database connection parameters are read from environment variables with sensible local defaults, overridden by Docker Compose's values when run in the containerized environment. Raw data file paths are similarly configurable via environment variables rather than hardcoded.

Running the suite:

```
pip install -r requirements-test.txt -r etl/requirements.txt
pytest tests/ -v
```

System tests auto-skip if their respective preconditions (exported CSVs, or Docker plus the raw data) aren't present, so the suite still runs (unit + integration only) on a checkout without either.

5. Test Execution Plan

Tied to the project's own weekly schedule rather than an invented cadence. Ownership below is stated honestly: no RACI document covering testing exists in this repository. A single reference

in the project's client-briefing document assigns "master database and data pulls" to two named team members — the closest existing scope to this pipeline's code — but nothing formally assigns test ownership specifically. Where a name is used below, it is inferred from that one reference, not from a formal assignment, and flagged as its own action item rather than assumed.

Phase	When	What Runs	Who
Unit + integration, every commit	Ongoing, starting Week 4	Unit and integration test files	Whoever touches ETL code — run locally before pushing; no CI gate yet (see Defect Management)
System test, before every new versioned dataset	Whenever a new dataset version is cut	CSV system test suite	Whoever cuts the version
Full Docker system test	Before any DB-schema-affecting change ships; verified passing 2026-07-17	Docker system test (Compose stack, ETL run, row-count cross-check) — auto-skips on machines without Docker/the raw data	Closest existing scope reference: the two team members named for "master database and data pulls" — needs explicit confirmation, not assumed
Validation-specific testing	Week 7 (per the project's own schedule)	Train/test/validation split checks, overfitting/underfitting checks — not yet built	Not assigned — no scope reference covers modeling/validation specifically; needs a decision
UAT round 1	As soon as the client has bandwidth	Structured review of both data dictionaries and the modeling dataset	Project client / client team
UAT round 2 (peer team)	Before Week 9 presentation prep	Independent reproduction of one pipeline run and review of methodology docs for clarity	Second student team + our team

Dependencies: system tests depend on the exported data being current (regenerated via the pipeline chain before running); the Docker system test runs today on any machine with Docker and the raw data present, and a fresh clone or CI runner now has a documented manual download step to follow first (Section 4); UAT round 2 depends on round 1 feedback being incorporated first, so the peer team is not reviewing something already known to be wrong.

Action item surfaced by writing this plan: a real RACI covering test ownership specifically does not exist yet. Recommend the team produce one, even a short one, rather than this plan inferring ownership from a single unrelated line item.

6. Defect Management

No formal issue tracker is wired up yet — this section states the process going forward, not one already running.

Reporting

Defects found via testing (automated or manual/UAT) are filed as GitHub Issues, tagged by the pipeline stage affected. Each issue states: what broke, the failing test (if automated) or the specific input that triggered it, and expected vs. actual output.

Severity tiers

Tier	Definition	Response
Blocking	Breaks a pipeline stage outright (e.g., a missing dependency)	Fix before the next system test run
Data-quality	Produces output but the output is wrong (e.g., a join bug producing duplicate or misattributed rows)	Fix before the next versioned dataset cut, documented in the relevant memo regardless of fix timing
Documentation	Code and docs disagree but the code itself is fine	Low priority, batched into the next doc-touching commit

Tracking and resolution

Issue status lives on the GitHub Issue itself, with no separate spreadsheet, to avoid a two-sources-of-truth problem. Every defect fix ships with a regression test named for the bug — a defect is not considered resolved until it is resolved and covered by a test that would fail if it recurred.

7. Regression Testing

- When: the full unit and integration suite runs before every commit that touches pipeline code, not on a fixed calendar cadence — appropriate for a small team iterating quickly rather than a large one needing a scheduled batch window.
- What: every previously found bug in this pipeline has a corresponding test — regression here specifically means "did a fixed bug come back," not just "did anything change."
- Full-pipeline regression: the system test suite reruns the entire CSV-driven pipeline chain against current data whenever any script in that chain changes, catching regressions that only manifest at real production scale. On machines with Docker and the raw data available, the Docker system test extends this to the database-backed join functions too, cross-checking the DB path's row counts against the CSV path's — not yet possible in CI.

No CI yet: this is a real gap, not glossed over. Recommendation: wire up a continuous-integration workflow to run the non-Docker tests on every push once the team has bandwidth, so regression testing stops depending on individual discipline.

8. Documentation and Reporting

- Format: test results are reported as dated updates in the relevant project documentation, not as a separate test-report artifact disconnected from the data findings they relate to. This test plan is the canonical reference for test process; individual results stay attached to the deliverable they validate.
- Frequency: after every system-test run that precedes a new versioned dataset, and immediately whenever a defect is found (not batched).
- Recipients: a client-facing briefing document is the established channel for anything that affects data the client would review — test and defect findings relevant to them are added there specifically, not just to internal technical memos.
- UAT reporting: feedback from the client and the peer team is logged in that same client-facing document's update sections, so UAT feedback and technical defect tracking do not fragment into separate, hard-to-reconcile systems.

9. Sample Test Report

Per the assignment instructions ("test reports… with sample/test data, inputs, and expected outputs"): the run below is real, not illustrative — captured 2026-07-17 against this project's actual code and data, not a mocked-up example.

Automated suite — full run

```
$ pytest tests/ -v
...
57 passed in 115.31s (0:01:55)
```

All 57 tests across all six test files passed: test_build_features.py (21), test_build_modeling_dataset.py (7), test_build_rigor_classification.py (11), test_combine_schools.py (9), test_docker_pipeline.py (2), test_integration_pipeline.py (2), test_system_pipeline.py (5). This run included the Docker-gated tests, meaning the raw source data was present and the full Postgres pipeline was exercised, not just skipped.

Sample test case, executed — UT-01 (private-school sector classification)

Field	Value
Test data / input	A fixture row: school_id="S2", pss_id="P1", nu_type=NaN, school_level=NaN (see tests/conftest.py)
Action	build(tiny_schools_org_all)
Expected output	is_private_hs == True, sector == "private"
Actual output	is_private_hs == True, sector == "private"
Result	PASS

Sample test case, executed — ST-02/ST-03 (Docker/Postgres system test)

Field	Value
Test data / input	Raw source data under data/updated-sheng/ (~2.6 GB — see Section 4); a freshly created Postgres 16 container
Action	docker compose up -d db, wait healthy, docker compose build etl, docker compose run --rm etl, then query schools_org_all row count
Expected output	Exit code 0; "Pipeline completed successfully" in output; schools_org_all row count matches the CSV-path result (53,966)
Actual output	Exit code 0; pipeline completed in ~106 seconds; schools_org_all row count = 53,966, exactly matching the CSV path
Result	PASS

Sample query against the live database — real data, not a mock

Demonstrates the database is genuinely queryable end to end, using two arbitrary example queries against schools_org_all (53,966 rows) in the running Postgres container:

Query 1 — schools with the highest AP-test-taking intensity:

```
SELECT school_name, state, nu_avg_num_ap_tests_taken, nu_avg_freshman_sat
FROM schools_org_all
WHERE nu_avg_num_ap_tests_taken IS NOT NULL
ORDER BY nu_avg_num_ap_tests_taken DESC
LIMIT 5;
```

School	State	Avg AP tests/student	Avg freshman SAT
School of Science and Engineering	TX	15.49	1290
School for the Talented and Gifted	TX	14.16	1250
BASIS Chandler	AZ	12.33	1260
BASIS Peoria	AZ	11.76	1200
IDEA College Preparatory Pharr	TX	11.58	1080

Query 2 — AP course counts and graduation rates by state (public HS only, states with 500+ schools):

```
SELECT state, COUNT(*) AS n_schools,
       ROUND(AVG(crdc_ap_courses)::numeric, 1) AS avg_ap_courses_offered,
       ROUND(AVG(grad_rate_2021)::numeric, 1) AS avg_grad_rate
FROM schools_org_all
WHERE school_level = 'High'
GROUP BY state HAVING COUNT(*) > 500
ORDER BY avg_grad_rate DESC LIMIT 5;
```

State	# Schools	Avg AP courses offered	Avg grad rate
TX	1,959	12.5	88.8%
PA	716	11.0	88.5%
WI	560	10.0	87.9%
MO	658	9.8	87.6%
NY	1,182	10.9	86.5%

Worth noting honestly rather than cropping out of the sample: this same query returns a null average grad rate for Washington state (657 schools) — not a bug, a real coverage gap already documented in the data dictionary (EDFacts grad-rate data isn't complete for every state). Including it here is deliberate: a sample test report that only shows clean rows isn't an honest sample.